2023 IEEE/IFIP Network Operations and Management Symposium (NOMS 2023) - Shortpaper

A Novel Poisoning Attack on Few-Shot based Network Intrusion Detection

Nour Alhussien
School of Computer and Cyber Sciences,
Augusta University
Augusta, Georgia, USA
noalhussien@augusta.edu

Ahmed Aleroud
School of Computer and Cyber Sciences,
Augusta University
Augusta, Georgia, USA
aaleroud@augusta.edu

NOMS 2023-2023 IEEE/IFIP Network Operations and Management Symposium | 978-1-6654-7716-1/23/$31.00 ©2023 IEEE | DOI: 10.1109/NOMS56928.2023.10154453

***Abstract*—**With the advancement of Machine Learning (ML) algorithms, more organizations started using Machine Learning based Intrusion Detection Systems (ML-IDSs) to mitigate cyberattacks. However, the lack of training datasets is a major challenge when creating those systems. Therefore, using pre-trained models and small amount of labeled network data or few-shots from internal sources are possible solutions to overcome this challenge. However, using pretrained models or external datasets introduces the risk of poisoned machine learning models. This work investigates a novel poisoning attack that creates a diverse mini cluster of attacks and normal instances around an attack instance, then use the instances in that cluster to poison that instance. The poisoned instances are then injected into training data. A trained model is then created by projecting a labeled data from a poisoned source and the few labeled shots from the target organization. An anomaly-based intrusion detection model is utilized to examine the effectiveness of the introduced approach under the proposed poisoning attack. The results have shown that the attack is effective in the context of few-shot IDS learning.

Keywords—Intrusion detection, few-shot learning, poisoning attacks, anomaly detection, machine learning.

I. INTRODUCTION

Modern Intrusion Detection Systems (IDSs) analyze the content of network packets to predict malicious activities [1, 2]. With the advancement of Machine Learning (ML) models, many IDSs utilize such models in their processes. However, the main challenge remains to obtain representative datasets to create those models. Organizations started to use pre-trained models, partially trained models, or even external data sources to train their own IDSs. However, pre-trained models are more vulnerable to adversarial attacks because they have already been exposed to a wide range of input data and may include known patterns that can be exploited by an attacker to compromise the integrity of the ML models [3]. Supply chain attacks and data poisoning to ML algorithms are unfortunately a thing. Typically, poisoning occur when adversaries provide a stream of bad information by flooding a target organization with a malware designed to refine ML detection away from the techniques they use for the main attack. Recent research by SANS Institute, shows that data poisoning attacks against ML used in security systems including network security systems may be attackers' next big vector [4]. Existing research has demonstrated the feasibility of poisoning attacks on ML systems, deep learning models, and ML-IDSs. However, with the trend of fine-tuning IDSs based on pre-trained models and using external data sources, there is a need to investigate if poisoning attacks on external training sources leads to a misbehaving target model. Recent approaches focus on injecting poisoned samples that are similar to the highly ranked training sample [5]. Highly ranked samples are network flows that have a good power to differentiate between attacks and benign activities. Those flows contain network features with good information gain or low entropy. The assumption in such approaches is that using highly ranked training instances from a benign class to create poisoned attack instances can lead to higher evasion rates. Such instance similarity-based attacks utilize similarity with unlike-similar neighbors to create the attacks. The rationale is that since highly ranked training samples influence a ML model's decision, injecting poisoned samples close (in the feature space) to the highly ranked instances affects the training process by propagating the labels of the poisoned samples to their influential neighbors, thus, degrades the resulting model's performance. This assumption, while valid, the success of such attacks depends solely on the number of the poisoned instances around the highly ranked benign instances. As such, in the related research in [5], when adding two poisoned instances around each target sample, the attack success rate was significantly lower. However, increasing the percentage of poisoned data makes the attack easy to detect using anomaly detection techniques. Furthermore, this leads to a model that votes most of the time to the majority class, therefore, overfitting is very likely.

We propose a double nearest neighbor poisoning approach, which creates a mini cluster around an attack instance to poison it. The cluster contains the nearest neighbors from the between-class instances (highly ranked benign instances) and from the within-class instances (low ranked attack instances). The adversary poisons the instances using a perturbation function that adds noise utilizing both nearest neighbors. The instances used for poisoning are the most/least influential ones. To selects those instances; we used the Support Based Bayesian Ranking Algorithm (SBBR). In addition to using the most influential benign instances to poison an attack instance, we used low ranked attack instances, which are mostly edge instances that a ML model finds anyway difficult to classify, therefore, poisoning an attack instance using those instances is one approach that leads to higher evasion rate as demonstrated in our experiments.

We are motivated by studying the robustness of ML-IDSs that are trained on external data sources. Our assumption is that the system will be trained on a few shots of an internal data which has a different dimensionality, feature spaces, or even attack signatures compared to external sources. Our assumption is valid due to the complexity of creating realistic

This material is based upon work supported by the National Science Foundation Grant No. 2131538

Authorized licensed use limited to: Telecom SudParis. Downloaded on October 27,2026 at 10:22:54 UTC from IEEE Xplore. Restrictions apply.

intrusion detection datasets. Our research question is to examine *if such transfer learning approaches are robust against our proximity-based poisoning attack*.

The contributions of this work are as follows: to the best of our knowledge, this is the first attempt to investigate targeted poisoning attacks in anomaly-based intrusion detection approaches created based on few-shot learning. Second, we designed a targeted poisoning attack that inherits the features of the low ranked similar neighbors and the highly ranked unlike nearest neighbors. The rest of this paper is organized as follows: Section II discusses the related work. Section III presents our approach and threat model. Section IV discusses the results of our experiments. The research is concluded in section V.

II. RELATED WORK

The existing research investigates the feasibility of poisoning attacks on different cyber domains such as worm signature generation [6], spam filters [7], graph-based classification methods [8], support vector machines [9] and behavioral malware clustering [10]. Based on attacker goals, poisoning attacks are classified into availability attacks and integrity/targeted attacks [11,12]. Other classification of attacks includes classifying adversarial attacks are classified into training time attacks [5, 13, 14] and testing time attacks [15, 16]. In all such attacks, the attackers' capabilities vary while compromising the target model. For example, when the attackers have power on feature sets, they can deceive the target model either by adding noise to features or through label flipping [17]. The former uses perturbation functions to modify the training data without changing labels which is known as clean label poisoning attacks [16].

The limited availability of new labeled attacks and malware data leads to adapting transfer learning (TL) to overcome this challenge [18]. TL boosts the robustness of ML models through extending the knowledge learned from a known source model into a target model without requiring the two models to have a similar feature space. One variation of TL is Few-shot learning (FSL). There has been some recent research on creating poisoning attacks that target FSL models such as images and visual object tracking [19,20].

III. APPROACH

A. Threat Model

Context: Our goal is to create a ML-ID model using one or more data sources and partially target's labeled data, then test that model on a single target T. We utilize training time poisoning attack, i.e., the adversary interferes with the training data to inject the poisoned examples. To implement our attack, we assume that the adversary has compromised one or more sources containing training data following the assumptions in previous works on poisoning attacks [3]. We consider a scenario in which one of the sources $\{S_1, S_2, \ldots,$ and $S_n\}$ is poisoned by a malicious sample when there is one target T. We assume that data sources consist of both benign and malicious flows with a smaller fraction of malicious instances compared to benign activities.

Attacker knowledge and capability: we assume that the attacker has access to manipulate and poison small fraction of training data on a data source. This attack is also possible by exploiting the victim's data supply chain, supplying them with a poisoned data that remains undetected during training and evaluation. The adversary is assumed to have a partial knowledge about features of the target data, which enables him to create a poisoned example based on features that co-exist in the target data.

Adversary's Goal: The adversary goal is to significantly degrade the True Positive Rate (TPR) of the classification model m. While poisoning attacks can be designed to achieve different objectives, our approach focuses on degrading the TPR.

B. A few-shot Intrusion Classification Model

Our approach works as follows: Source (i.e., external) datasets are the sources of network data to train a classification model m. Each source is denoted by S, such that S_i contains a dataset with instances and features. The features represent the traffic patterns of network data, each of them is denoted by x_s^i. Instances represent NetFlows, where each flow has a label y_s^i as a benign activity or an attack. The target (i.e., internal) dataset T is represented by a set of features each of which is denoted by x_t that are fed as inputs to an intrusion detection system to predict the class label y_t, formally $m: f(x) = y$. We assume that T has few labeled flows that can be used along with the data from S to train a model m, where T may share common features with S and S is vulnerable to poisoning attacks. We created a classification model through a unified projection of sources using Laplacian and diagonal matrix decomposition. Under these settings an adversary will inject a poisoned data in the source domain datasets before the projection step and evaluate its success in the target domain.

Formally, suppose we have a ML-IDs that is trained on one or more sources, and one target. The sources are denoted by $S_i = \{S_1, S_2, \ldots, S_n\}$ where S_i denotes the ith source in S and n is the number of sources. Each source contains features which can be represented as $S_i = \{x_1^{si}, x_2^{si}, \ldots, x_j^{si}\}$ whereas x_k^{si} is the value of kth feature in S_i and j is the dimensionality of S_i. Each source contains data with labels $Y_i^s = \{y_i^s\}$ such that each $y_i^s \in \{attack, benign\ activity\}$. The target T consists of limited number of labeled training flows T^{tr} and testing flows T^{ts}. The labeled data in the target domain is denoted by T_l^{tr}, such that $T_l^{tr} = \{x_1^{tr}, x_2^{tr}, \ldots, x_m^{tr}\}$, where x_m^{tr} and m are feature values in T_l^{tr} and the dimensionality of T_l^{tr} respectively. T_l^{tr} has few labeled flows denoted by $Y_l^{tr} = \{y_{l1}^{tr}, y_{l2}^{tr}, \ldots, y_{ln_l}^{tr} |\ n_l\}$ where n_l is the number of labeled flows in T_l^{tr}. T_u^{ts} denotes the testing data samples that has unlabeled flows and is denoted by $Y_u^{ts} = \{y_{u1}^{ts}, y_{u2}^{ts}, \ldots, y_{un_u}^{ts} |\ n_u\}$ where n_u is the number of unlabeled flows in T_u^{ts}. The test data T^{ts} contains attacks that may or may not exist in the training data T_l^{tr}. Our objective is to create a common space V_{si}^t for one or more sources and the small percentage of labeled target data , then train V_{si}^t to predict unlabeled data in T. A good ML-IDS tries to find the optimal dimensionality of V_{si}^t. We constructed a unified projection V_{si}^t for single/multi sources in $S_i = \{S_1, S_2, \ldots, S_n\}$ and the labeled data in T, using the projection approach in [21]. Considering the diverse dimensionality between each source and target, we applied

Authorized licensed use limited to: Telecom SudParis. Downloaded on October 27,2026 at 10:22:54 UTC from IEEE Xplore. Restrictions apply.

some preprocessing on the data for each source to rescale the features into a common scale using z-score normalization while maintaining the differences in the ranges of values as $V_s^n = \{V_s^1, V_s^2, \ldots, V_s^n\}$ and V_t respectively. Then, we create a block diagonal matrix V_s^t which first aligns V_s^1 and V_t as inputs matrices into the diagonal of V_s^t. To preserve local geometrical structures of each source, we generate the geometry Laplacian Matrix using the degree matrix D of V_s^t and its block diagonal matrix. To extract a unified feature representation, in lower dimensional space, we applied general eigen decomposition to V_s^t then multiply it by the geometry Laplacian to generate CV_s^t matrix, CV_s^t is supposed to preserve correlations in the original sources. Singular value decomposition is applied to CV_s^t, resulting in a diagonal matrix DD that contains the eigenvalues, and the matrix P, which corresponds to the right eigenvectors of V_s^t. The resulting projected dimensionality RD is identified using a user defined threshold ∂ (i.e., set to 0.15 in our experiment) that represents the percentage of the dimensionality of the target T to be included in projected space Z as shown in the following equations:

$$RD = T.\partial \quad (1)$$

$$P = PP(All_{Rows}, Col_1, \ldots, Col_{RD}) \quad (2)$$

$$Z = P'.V_s^t \quad (3)$$

C. Finding Influential Instances

The selection of influential instances relies on determining the attribute values that are more present in specific class rather than other classes. Inspired by the authors in [21, 22] Support Based Bayesian Ranking Algorithm (SBBR) can find those critical values. The process is conducted by mapping attributes' values into bins using equal frequency binning, then, finding the weight for each attribute or attribute bins using Bayesian Ranking Measure, which indicates the probability for attribute to be belonging to one as opposed to other classes. We here refer to the benign class as $Target$ and the attack class as $Rest$. The main objective of our poisoning attack is to use the most influential (highly ranked) instances from the *Target* class and the least influential instances (least ranked) from the rest class to poison an attack instance, so that it can be classified as benign at testing time. Bayesian ranking calculates the likelihood for each class using the evidence E and a prior probability $pr(H)$ for the hypothesis $H \in \{Target, Rest\}$ using Equation 4 as follows:

$$W(Target|E) = \frac{pr(Target|E)^2}{pr(Target|E) + pr(Rest|E)} \quad (4)$$

For the dataset $D = \{A_i: (x_{si}^t, y_i)\}_{i=1}^k$, A_i is the ith attribute in D, $x_{si}^t \in V_s^t$ are the common features between S and T, and $y_i \in Y = \{Target, Rest\}$ is the label for each flow| $\forall A_i \; \exists b_i \in B = \{b_i\}_{i=1}^n$, b_i are the equivalent sets bins. The main goal of this technique is to find the weight for each bin b_i in each attribute A_i, then the weight for each instance is calculated by computing the product of each bin's weight. The instances with the highest weights are considered the most influential instances and instances with the lower weights are the less influential ones.

Algorithm 1 shows the details for the selection of most influential instances. $\bar{I}$ contains the most influential instances in the target class. $\bar{\bar{I}}$ contains the least influential instances from the rest class. B is the number of bins for each feature value. P is the percentage of influential instances, m is the number of values in each feature, and n is the number of values in each bin b_i, such that $n = \frac{m}{|B|}$.

Algorithm 1. Finding Influential Instances

Input: D, B and P
Output: $\bar{I}, \bar{\bar{I}}$

For each instance x_{si}^t in D
1: **For** $i = 1$ to $|A|$
2: Find m and n
3: **For** $k = 0$ to B
4: **For** $j = (k * n)$ *to* $((k+1) * n)$
5: $W \leftarrow$ Find the weight using Equation (4)
6: **End for**
7: $U \leftarrow$ Find unique weight for each class in W
8: **End for**
9: Calculate the product of weights $wp_{x_{si}^t}$ in U
10: **End for**
11: Add $wp_{x_{si}^t}$ to the ranking matrix R
12: **End For**
13: Find instances $\bar{I}, \bar{\bar{I}}$ in R with the highest/lowest weights using a threshold P
14: **Return** $\bar{I}, \bar{\bar{I}}$

D. Data Poisoning

The existing techniques apply specific noise with label flipping on nearest unlike neighbor (NUN) instances [5]. However, our approach applies poisoning to an instance using both highly ranked unlike nearest (NUN) instances and low ranked similar nearest (NSN). The poisoned instance is created as follows: For an instance $x_{si}^t \in D$, we separately select x_{rest}^{nsn} which is a low ranked NSN instance labeled as $Rest$ and x_{target}^{nun} which is a high ranked NUN instance labeled as $Target$, x is then poisoned as follows [21]:

$$x^p = x + (x - x_{rest}^{nsn}) * \epsilon_1 - sign(\epsilon_1)\left(x - x_{target}^{nun}\right) * |\epsilon_2| \quad (5)$$

Where x^p is the poisoned instance, ϵ_1 and ϵ_2 represent poisoning tuning parameters | $0 > \epsilon_1, \epsilon_2 < 1$. Existing research found that ϵ_2 needs to be higher than ϵ_1. In adversarial research those parameters can take a range of values between 0-1 but we found that most researchers use the values between 0.007 to 0.02 [23, 24].

IV. Experiments

A. Datasets

We conducted experiments on two network intrusion detection datasets CIC-IDS2018 [25] and UNSW-NB15[26]. Table I shows the details for the datasets used in this research, whether they were used as source or target, and the sampled attacks and benign activities.

TABLE I. DATASETS USED FOR EXPERIMENTS

Criteria	Source (S)	Target (T)
Dataset	UNSW	CIC-IDS2018
Number of Features	43	82
Number of Flows	20803	14310
Attack Samples	2766	6641
Benign Samples	18037	7669

Authorized licensed use limited to: Telecom SudParis. Downloaded on October 27,2026 at 10:22:54 UTC from IEEE Xplore. Restrictions apply.

B. Experiment settings

Experiments were conducted on an iMac system with Intel(R) Xeon(R) W-2150B CPU of 3.00GHz and 64GB of RAM. The proposed learning models were implemented in Matlab. We used 5% of the training flows as poisoned sample in the source data, but no flows were poisoned in the target. We reported different intrusion detection metrics before and after poisoning such as AUC, G-MEAN, Accuracy (AUC), True Positive Rate (TPR), and False Positive Rate (FPR). Logistic regression was used as a classification approach based on a sigmoid function and a 0.5 anomaly score to classify an instance as an attack.

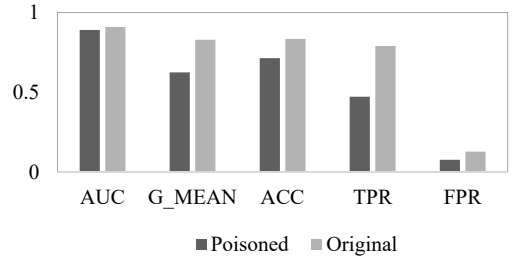


Fig. 1. Intrusion detection metrics before and after data poisoning (percentage of labeled shots from target = 0.1)

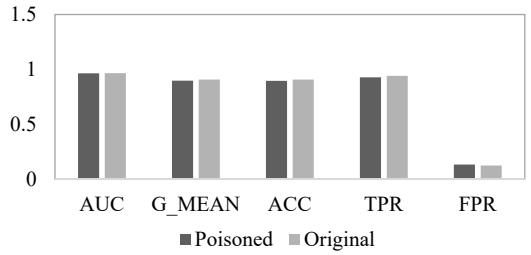


Fig. 2. Intrusion detection metrics before and after data poisoning (percentage of labeled shots from target = 0.2)

Fig. 1 Shows the results when 0.1 labeled instances were used from the target T. The most impacted metric is the TPR which declines from 79% to 47% indicating that the attack was significantly effective to evade the classifier at testing time. There is a minor impact on FPR, this is expected since the poisoning process will still produce attack instances that look like the target benign class. Therefore, the model sometimes misbehaves on few benign instances leading to classifying them as attacks, but this was limited in our experiment since the FPR is consistent before and after poisoning.

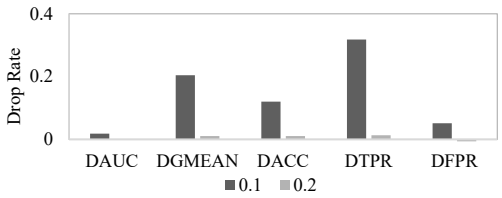


Fig. 3. Drop rate in intrusion detection metrics.

We noticed a limited impact of the attack when doubling the percentage of labeled shots as shown in Fig. 2. This can be a method to mitigate zero-shot or few-shot adversarial attacks. Fig. 3 Summarizes the impact of the attack on different metrics. The attack seems to have the highest impact on TPR compared to other metrics which is consistent with our attack goal. Nevertheless, the attack has also impacted other classification metrics.

C. Single Nearest Neighbor Compared to Double Nearest Neighbors Attack

This experiment aims at comparing our approach with those attacks that rely on instance power and unlike nearest neighbors (single nearest neighbors (SNN)), compared to our approach, which uses double nearest neighbors (DNN) as a cluster to poison an instance. This experiment is based on 0.1 shots for both approaches. It worth mentioning that our attack settings are different since it mainly focuses on fooling the classifier to classify attacks as benign. So, its objective is to minimize the TPR compared to the attacks that focuses on misclassification regardless of the predicted label [5]. Furthermore, the attack in [5] was implemented on a traditional machine learning settings not few-shot learning.

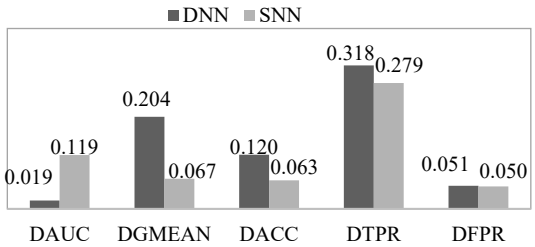


Fig. 4. Comparsion with attacks relying on nearest instances power (D stands for difference Before vs. after Attack)

Fig. 4 shows that our attack leads to higher evasion rate compared to SNN attacks in most of the classification metrics. Specifically, Accuracy and AUC. We did not notice a significant difference between our attack and other attacks based on FPR. In the last experiment we demonstrate how our attack behaves when changing the projected dimensions in the resulting training data. We varied the dimensionality after poisoning and used 0.1 as a percentage of labeled data from the target. Fig. 5 shows no significant difference in the attack effectiveness when dimensionality is changed, which indicates that the attack is still able to evade a classification model even when enough number of dimensions are used for prediction.

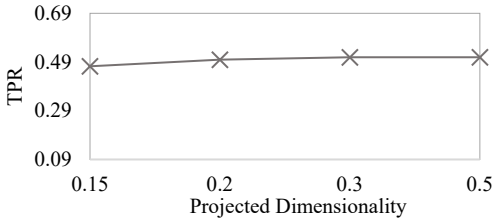


Fig. 5. Changing dimensionality vs. attack effectiveness

V. Conclusions and Future Work

This work investigates the effectiveness of a new poisoning attack on few-shot intrusion detection systems. Our attack is conducted by poisoning a model using the most influencing benign instances and the least influencing attack instances. Both sources and the labeled target data can be projected in the same sub-space and used to predict attacks at testing time. We show that our attack is effective enough in terms of reducing the TPR and other intrusion classification metric. We show that one way to defend against such attack is to increase the percentage of labeled shots from internal sources. We believe that validating our findings on larger datasets and increasing the number of poisoned sources is a potential future work.

Authorized licensed use limited to: Telecom SudParis. Downloaded on October 27,2026 at 10:22:54 UTC from IEEE Xplore. Restrictions apply.

VI. REFERENCES

[1] A. S. Dina and D. Manivannan, "Intrusion detection based on Machine Learning techniques in computer networks," Internet of Things, vol. 16, p. 100462, 2021.

[2] D. Chou and M. Jiang, "A survey on data-driven network intrusion detection," ACM Computing Surveys (CSUR), vol. 54, no. 9, pp. 1-36, 2021.

[3] T. Gu, B. Dolan-Gavitt, and S. BadNets, "Identifying vulnerabilities in the machine learning model supply chain," in Proceedings of the Neural Information Processing Symposium Workshop Mach. Learning Security (MLSec), 2017, pp. 1-5.

[4] J. Gregory "Data Poisoning: The Next Big Threat". Security Intelligence, https://securityintelligence.com/articles/data-poisoning-big-threat/ (accessed Sep. 26, 2022)

[5] S. Venkatesan, H. Sikka, R. Izmailov, R. Chadha, A. Oprea, and M. J. De Lucia, "Poisoning Attacks and Data Sanitization Mitigations for Machine Learning Models in Network Intrusion Detection Systems," in *MILCOM 2021-2021 IEEE Military Communications Conference (MILCOM)*: IEEE, pp. 874-879.

[17] H. Debar, M. Dacier, and A. Wespi, "A revised taxonomy for intrusion-detection systems," in *Annales des télécommunications*, 2000, vol. 55, no. 7: Springer, pp. 361-378.

[10] M. Roesch, "Lightweight intrusion detection for networks," in *LISA'99: Proceedings of the 13th USENIX conference on System administration, Nov.*, 1999, pp. 229-238.

[11] S. Rezaei and X. Liu, "A target-agnostic attack on deep models: Exploiting security vulnerabilities of transfer learning," *arXiv preprint arXiv:1904.04334,* 2019.

[6] L. Cavallaro, A. Lanzi, L. Mayer, and M. Monga, "LISABETH: automated content-based signature generator for zero-day polymorphic worms," in *Proceedings of the fourth international workshop on Software engineering for secure systems*, 2008, pp. 41-48.

[7] B . Nelson *et al.*, "Exploiting machine learning to subvert your spam filter," *LEET,* vol. 8, pp. 1-9, 2008.

[8] B. Wang and N. Z. Gong, "Attacking graph-based classification via manipulating the graph structure," in *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*, 2019, pp. 2023-2040.

[9] B. Biggio, B. Nelson, and P. Laskov, "Poisoning attacks against support vector machines," *arXiv preprint arXiv:1206.6389,* 2012.

[10] B. Biggio *et al.*, "Poisoning behavioral malware clustering," in *Proceedings of the 2014 workshop on artificial intelligent and security workshop*, 2014, pp. 27-36.

[11] B. Zhao and Y. Lao, "Towards Class-Oriented Poisoning Attacks Against Neural Networks," in *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, 2022, pp. 3741-3750.

[12] M. Jagielski, A. Oprea, B. Biggio, C. Liu, C. Nita-Rotaru, and B. Li, "Manipulating machine learning: Poisoning attacks and countermeasures for regression learning," in *2018 IEEE Symposium on Security and Privacy (SP)*, 2018: IEEE, pp. 19-35.

[13] P. Li, Q. Liu, W. Zhao, D. Wang, and S. Wang, "Chronic poisoning against machine learning based IDSs using edge pattern detection," in *2018 IEEE International Conference on Communications (ICC)*, 2018: IEEE, pp. 1-7.

[14] J. Xu, Y. Wen, C. Yang, and D. Meng, "An approach for poisoning attacks against rnn-based cyber anomaly detection," in *2020 IEEE 19th International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom)*, 2020: IEEE, pp. 1680-1687.

[15] K. Talty, J. Stockdale, and N. D. Bastian, "A Sensitivity Analysis of Poisoning and Evasion Attacks in Network Intrusion Detection System Machine Learning Models," in *MILCOM 2021-2021 IEEE Military Communications Conference (MILCOM)*, 2021: IEEE, pp. 1011-1016.

[16] D. Zügner, A. Akbarnejad, and S. Günnemann, "Adversarial attacks on neural networks for graph data," in *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2018, pp. 2847-2856.

[18] A. Singla, E. Bertino, and D. Verma, "Overcoming the lack of labeled data: Training intrusion detection models using transfer learning," in *2019 IEEE International Conference on Smart Computing (SMARTCOMP)*, 2019: IEEE, pp. 69-74.

[19] E. T. Oldewage, J. F. Bronskill, and R. E. Turner, "Attacking Few-Shot Classifiers with Adversarial Support Sets," 2020.

[20] Y. Li, H. Zhong, X. Ma, Y. Jiang, and S.-T. Xia, "Few-shot backdoor attacks on visual object tracking," *arXiv preprint arXiv:2201.13178,* 2022.

[21] H. Zhang, X. Zhu, Z. Li, B. Xu, X.-Y. Jing, and S. Ying, "On the Multiple Sources and Privacy Preservation Issues for Heterogeneous Defect Prediction," 2017.

[22] F. Peters, T. Menzies, L. Gong, and H. Zhang, "Balancing privacy and utility in cross-company defect prediction," IEEE Transactions on Software Engineering, vol. 39, no. 8, pp. 1054-1068, 2013.

[23] P. Papadopoulos, O. Thornewill von Essen, N. Pitropakis, C. Chrysoulas, A. Mylonas, and W. J. Buchanan, "Launching adversarial attacks against network intrusion detection systems for iot," *Journal of Cybersecurity and Privacy,* vol. 1, no. 2, pp. 252-273, 2021.

[24] I. J. Goodfellow, J. Shlens, and C. Szegedy, "Explaining and harnessing adversarial examples," *arXiv preprint arXiv:1412.6572,* 2014.

[25] A Realistic Cyber Defense Dataset (CSE-CIC-IDS2018), "Registry of Open Data on AWS", https://registry.opendata.aws/cse-cic-ids2018. (accessed Sep. 26, 2022).

[26] N. Moustafa, J. Slay, and G. Creech, "Novel geometric area analysis technique for anomaly detection using trapezoidal area estimation on large-scale networks," *IEEE Transactions on Big Data,* vol. 5, no. 4, pp. 481-494, 2017.

Authorized licensed use limited to: Telecom SudParis. Downloaded on October 27,2026 at 10:22:54 UTC from IEEE Xplore. Restrictions apply.